

Université Paris-Saclay UFR des Sciences d'Orsay Computer Science Department
Master 2 Quantum and Distributed Computer Science (QDCS)
2025-26 Internship Report

BLOB COMPUTING IN HOMOGENEOUS ISOTROPIC DISTRIBUTED SYSTEMS

an internship authored by

Lucas Labouret

performed at the
Laboratoire Interdisciplinaire des Sciences du Numériques
and supervised by
Dr. Frédéric Gruau

10 July, 2026

Abstract

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Ut purus elit, vestibulum ut, placerat ac, adipiscing vitae, felis. Curabitur dictum gravida mauris. Nam arcu libero, nonummy eget, consectetur id, vulputate a, magna. Donec vehicula augue eu neque. Pellentesque habitant morbi tristique senectus et netus et malesuada fames ac turpis egestas. Mauris ut leo. Cras viverra metus rhoncus sem. Nulla et lectus vestibulum urna fringilla ultrices. Phasellus eu tellus sit amet tortor gravida placerat. Integer sapien est, iaculis in, pretium quis, viverra ac, nunc. Praesent eget sem vel leo ultrices bibendum. Aenean faucibus. Morbi dolor nulla, malesuada eu, pulvinar at, mollis ac, nulla. Curabitur auctor semper nulla. Donec varius orci eget risus. Duis nibh mi, congue eu, accumsan eleifend, sagittis quis, diam. Duis eget orci sit amet orci dignissim rutrum.

Keywords: word1. word2. word3. word4.

Contents

1	Introduction	1
1.1	Presentation of the host organization	1
1.2	Internship context and objectives	1
1.3	What is Blob computing ?	2
2	Performed work	5
2.1	Blob language and interpreter	5
2.1.1	The interpreter	6
2.1.2	The language	6
2.1.3	Basic instruction parallelization	8
3	Evaluation and discussion	9
3.1	Evaluation of the performed work	9
3.2	Impact of the internship	9
3.3	Reflections on the internship	9
3.3.1	Difficulties encountered and reactions	9
3.3.2	Skills acquired and lessons learned	9
3.3.3	Career plans after the internship	9
4	Conclusion	10
4.1	Acknowledgements	10
	References	10
	Appendix A built-in Blob instructions	11

Chapter 1

Introduction

1.1 Presentation of the host organization

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Ut purus elit, vestibulum ut, placerat ac, adipiscing vitae, felis. Curabitur dictum gravida mauris. Nam arcu libero, nonummy eget, consectetur id, vulputate a, magna. Donec vehicula augue eu neque. Pellentesque habitant morbi tristique senectus et netus et malesuada fames ac turpis egestas. Mauris ut leo. Cras viverra metus rhoncus sem. Nulla et lectus vestibulum urna fringilla ultrices. Phasellus eu tellus sit amet tortor gravida placerat. Integer sapien est, iaculis in, pretium quis, viverra ac, nunc. Praesent eget sem vel leo ultrices bibendum. Aenean faucibus. Morbi dolor nulla, malesuada eu, pulvinar at, mollis ac, nulla. Curabitur auctor semper nulla. Donec varius orci eget risus. Duis nibh mi, congue eu, accumsan eleifend, sagittis quis, diam. Duis eget orci sit amet orci dignissim rutrum.

1.2 Internship context and objectives

This year's internship is the continuation of my M1 internship (2025). Last year, I worked on developing a parallel algorithm for the simulation of some fundamental operations in the context of blob computing. This year's internship was split into three main parts:

- Specifying a language for the descriptions of Blob programs, and implementing an interpreter for this language making use of the parallel algorithm developed during my M1 internship.
- Building a graphical user interface to easily visualize the execution of Blob programs, and to allow users to interact with the simulation.
- Redevlopping Blob algorithms initially designed to work on a hexagonal lattice in the new language, and adapting them to work on a homogeneous isotropic medium.

The main objective of this internship is to explore the potential of blob computing in homogeneous isotropic distributed systems, and to contribute to the development of tools and algorithms for this computational paradigm. In particular, the aim was to design and implement an algorithm to solve the homogenization problem : given an initial set of particles in

a medium, we want to spread them in the medium so as to minimize the maximal distance between any two particles (or a particle and the border of the medium).

1.3 What is Blob computing ?

Blob computing is a computational paradigm that draws from cellular automatas. In blob computing, computation is performed by a set of simple, locally interacting agents, called "processing elements" (PEs). These PEs all execute the same sequence of operations synchronously. They are arranged in space following the topology of a Delaunay triangulated embedded planar graph. The set of all PEs is called a "computing medium", or simply "medium". The PEs are of different types, depending on their position in the graph. Each PE can only directly communicate with its neighbors in the graph, and the type of communication depends on the types of the PEs involved.

There are 9 distinct types of PEs :

- 3 simplicial types that correspond to the vertices, edges, and faces (excluding the exterior face) of the graph
- 6 transfer types. These are used to transfer information between the simplicial PEs. They are defined by the type of simplicial PE they are transferring from and to. For example, a transfer PE of type "Ve" ("vertex to edge") is used to transfer information from a vertex PE to an edge PE.

While the PEs are simple and have limited computational capabilities, they can interact with their neighbors to perform complex computations. PEs are restricted to a very limited set of operations.

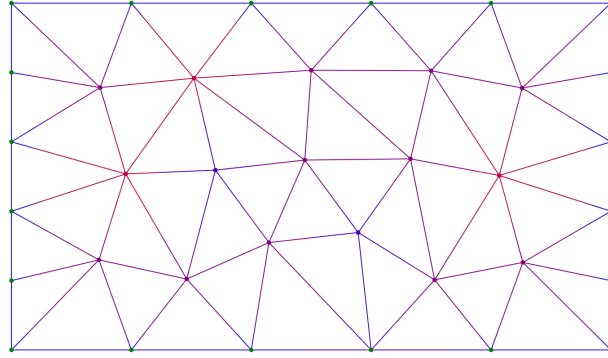
- **Broadcasts** : a simplicial PE can broadcast a message to its neighboring transfer PEs of a given type. For example, a *Vertex* can broadcast a message to its neighboring *Ves*, and a *Face* can broadcast a message to its neighboring *Fes*. Importantly, *this is the only way of direct communication from simplicial to transfer PEs*, it is not possible for a simplicial PE to communicate single out a transfer PE to communicate exclusively with.
- **Logical reductions** : a simplicial PE can perform a reduction operation on the messages received from its neighboring transfer PEs of a given type. For example, a *Vertex* can receive a message that is the result of a successive OR (resp. AND, XOR) of the messages sent by its neighboring *Ves*. Similarly to broadcasts, this is the only way of direct communication from transfer to simplicial PEs.
- **Transfers** : a transfer PE can send a message to its dual transfer PE. For example, a *Ve* can send a message to its dual *Ev*.
- **Rotations** : a simplicial PE can "rotate" a message clockwise or counterclockwise between its neighboring transfer PEs of a given type. For example, a *Vertex* can rotate a message from its *Ves* to its *Vfs* (and vice versa), and from its *Vfs* to its *Ves*. This is the only way of direct communication between transfer PEs of different types.

- **Logical operations** : any PE can perform logical operations (AND, OR, XOR) on any bits of its state.

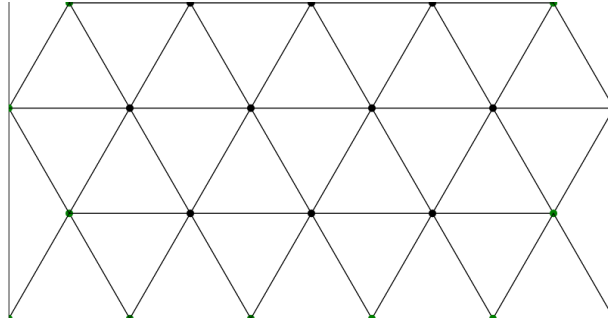
About the media

In this internship I focused on a very specific kind of medium : homogeneous and isotropic with a regular rectangular border. A medium is homogeneous if the vertices of the underlying embedded graph are distributed in a way that minimizes (or is close to minimizing) the maximal distance between any two vertices. It is isotropic if the direction of the edges follows no preferential direction (i.e. the distribution of the angles of the edges is close to uniform).

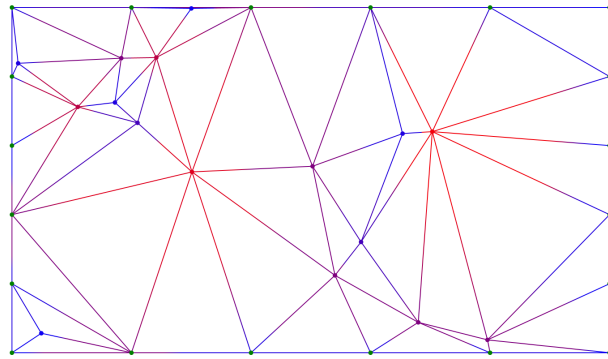
We accepted a small caveat regarding the homogeneity and anisotropy of the medium : we allowed the border to be a perfect rectangle with regularly spaced vertices. Such a border is very helpful in controlling strange behaviors that can arise near it, because it allows us to easily "trick" the border PEs into behaving as if there a perfect mirror image of the medium on the other side of the border. This mirror image is useful for reasons that will become clear later.



(a) Homogeneous and isotropic.



(b) Homogeneous but not isotropic.



(c) Isotropic but not homogeneous.

We chose to focus on this specific kind of medium for two reasons. First, it is a natural setting for many applications of blob computing, such as swarm robotics or distributed sensor networks. Second, it allows us to explore the potential of blob computing in a more realistic and complex setting than the hexagonal lattice, which was the focus of most of Frédéric's previous work in this area.

Chapter 2

Performed work

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Ut purus elit, vestibulum ut, placerat ac, adipiscing vitae, felis. Curabitur dictum gravida mauris. Nam arcu libero, nonummy eget, consectetur id, vulputate a, magna. Donec vehicula augue eu neque. Pellentesque habitant morbi tristique senectus et netus et malesuada fames ac turpis egestas. Mauris ut leo. Cras viverra metus rhoncus sem. Nulla et lectus vestibulum urna fringilla ultrices. Phasellus eu tellus sit amet tortor gravida placerat. Integer sapien est, iaculis in, pretium quis, viverra ac, nunc. Praesent eget sem vel leo ultrices bibendum. Aenean faucibus. Morbi dolor nulla, malesuada eu, pulvinar at, mollis ac, nulla. Curabitur auctor semper nulla. Donec varius orci eget risus. Duis nibh mi, congue eu, accumsan eleifend, sagittis quis, diam. Duis eget orci sit amet orci dignissim rutrum.

2.1 Blob language and interpreter

The blob language is a simple, high-level language that allows users to write Blob programs in a convenient way. It is based on the notion of field programming : primitive data types are maps from the PEs of a given type to values. Specifically, we only use boolean fields, which are maps from the PEs of a given type to boolean values. A Blob program is then a sequence of instructions that manipulate these fields. It is similar in spirit to an automaton, where the state of the medium at a given time is represented by a collection of fields, and the program defines a transition function that takes the current state as an input and produces the next state.

This field programming paradigm is very natural in the context of blob computing : since PEs act synchronously, any operation that can be performed by a PE can be easily expressed as an operation on the field of values of that PE type. For example, a rotation $Ve \leftarrow Vf$ can be expressed as an operation that takes the field of values of the Ves and produces a new field of values for the Vfs , where the value of each Vf is equal to the value of the Ve that is rotated to it. Similarly, a broadcast from *Vertex* to *Ves* can be expressed as an operation that takes the field of values of the *Vertices* and produces a new field of values for the *Ves*, where the value of each Ve is equal to the value of the *Vertex* that is broadcasting to it.

Because of the limited time available during this internship, I decided to develop a simple language that sits on top of Java. This allows us to avoid the overhead of developing a full-fledged programming language. In particular, this allows us to forgo the development of a lexer

and a parser, which would have been necessary if we had developed a standalone language. This also allows us to easily make the Blob language strongly typed object oriented, since we can directly use Java classes and objects in our Blob programs.

2.1.1 The interpreter

The interpreter is built around an Instruction interface, which declares a single method, `exec()`. Calling this method performs a single step of the instruction, then returns a boolean indicating whether the instruction has finished executing or not. The Instruction interface has two direct implementations : the BasicInstruction interface, which is implemented by single-step instructions, and the Procedure abstract class, which is used to create composite instructions. Basic instructions are the building blocks of Blob programs. They correspond to the basic operations that PEs can perform, such as broadcasts and transfers, as well as some interpreter specific operations, such as Show which marks a value to be displayed in the GUI, but otherwise has no effect on the state of the medium. Procedures are used to create more complex instructions by combining basic instructions and other procedures. They are implemented as a list of instructions, and the `exec()` method simply executes the instructions in order, moving on to the next instruction only when the current one has finished executing. Ultimately, the interpreter simply executes a given instruction on loop ad-vitam eternam, making the initial state of the medium (i.e. the initial values of the variables in the program) evolve according to the program.

It is important to note, however, that not all variables in the program are useful to describe the state of the program at the start of an iteration of the loop. Many variables are used temporarily to store intermediate results. The interpreter allows to mark those variables as temporary. The references to the fields contained in a temporary variable is set to *null* at the end of the procedure that created it, allowing Java's garbage collector to free the memory used by those fields. While this may not be important for small programs, given that the number of PEs in a medium can be very large, it is crucial to be able to free memory as soon as possible to avoid running out of memory.

2.1.2 The language

Since the Blob language is relatively simple, let's use a small example of a Blob program to illustrate its syntax and semantics. The following program implements a simple algorithm that allows a single starting particle to spread in the medium.

```
// A standard Java class
public class Grow {
    // A BoolVRef is a wrapper for a boolean field over the vertices.
    // Since the fields are meant evolve across iteration, this allows us to
    // keep a constant reference to pass around the program while new fields
    // replace the stale ones.
    // Here, we store the current spread.
    BoolVRef grow;
```

```

// The constructor is used to define the initial state of the program.
// Here, we call various methods which are not part of the Blob instruction set,
// such as Math.random().
// This is not an issue as the constructor is only executed once before the
// interpretation of the program, and does not interfere with the execution past
// the initial configuration.
public Grow(Medium medium) {
    int randIndex = (int) (Math.random() * medium.vertices.size());
    Vertex seed = (Vertex) medium.vertices.toArray()[randIndex];

    // Create a boolean over the vertices that is false everywhere
    BoolV seedField = BoolV.zeros();

    // Set a single vertex in the field (the seed) to true
    BoolV.setBit(seedField, seed, true);

    // Assigns the field to grow as the initial state of the program
    grow = new BoolVRef(seedField);
}

// A Blob method is defined as an inner class extending the Procedure abstract class.
// The method body is defined in the constructor of the class.
// This methods enables vertex-to-vertex communication by swapping to two ve's
// pointing to each edge.
private class Send extends Procedure {
    public Send(BoolVRef ve) {
        // Boolean fields used to store intermediate results.
        // The use of tmp(...) makes the fields contained in the refs open
        // to garbage collection once we finish executing this method.
        BoolEvRef ev = tmp(new BoolEvRef());
        BoolEfRef ef = tmp(new BoolEfRef());

        // Blob instructions
        // These methods add the corresponding instructions to the execution
        // list of this procedure.
        transfer(ve, ev);
        rotCW(ev, ef);
        rotCW(ef, ev);
        transfer(ev, ve);

        // Note : Similarly the outer class constructor, this is only
        // executed once before the interpretation of the program.
    }
}

```

```

        // This means that the ev and ef references are only created once
        // before the execution.
        // Additionally this also enable the use of non-Blob methods to
        // setup the initial state, which can be useful to define deirectly
        // here values that should be reused throughout the exection.
    }
}

// A small helper that skips the need to use "new" everytime we call the method.
private Procedure send(BoolVRef ve) { return new Send(ve); }

// The method that performs the spread.
private class Spread extends Procedure {
    public Spread() {
        BoolVRef start = tmp(new BoolVRef());
        BoolVRef ve = tmp(new BoolVRef());

        set(grow, start);
        broadcast(grow, ve);

        // Since the send method is user defined and not part of the built-in
        // instruction set, it needs to be passed through a call(...).
        call(send(ve));

        redOr(ve, grow);
        or(start, grow, grow);
    }
}

public Procedure spread() { return new Spread(); }
}

```

2.1.3 Basic instruction parallelization

The field programming paradigm forms the basis of the parallelization of basic instructions, which is a key feature of the interpreter. Since the number of PEs in a medium can be very large, it is important to be able to execute instructions in parallel as much as possible. This is achieved by expressing a field of boolean values as an array of integers, where each PE is assigned a single bit (instead of a full byte) representing its value (*note : it is possible for a bit to be unused, but all PEs must have exactly one corresponding bit, and no bit can be used by more than one PE*). With a clever organization of this array, it is possible to greatly reduce the number of operations needed to execute an instruction on all PEs of a given type, by performing bitwise operations on the integers in the array.

Chapter 3

Evaluation and discussion

3.1 Evaluation of the performed work

3.2 Impact of the internship

3.3 Reflections on the internship

3.3.1 Difficulties encountered and reactions

3.3.2 Skills acquired and lessons learned

3.3.3 Career plans after the internship

Chapter 4

Conclusion

4.1 Acknowledgements

Appendices

Appendix A - List of built-in Blob instructions

Special instructions

Instructions that do not affect the execution but can be used by the interpreter.

- **print(String s)**: print s to the console whenever it is reached.
- **show(Ref r)**: indicate that the value of r should be displayed by the UI.
- **snapshot()**: signal to the ui that the current display should be saved as an image.

Boolean instructions

Instructions that only affect boolean fields.

- **And(BoolField a, BoolField b, BoolField res)**: store a AND b in c.
- **Broadcast(BoolFieldS a, BoolFieldT res)**: Broadcast a to res.
- **If(BoolField cond, BoolField t, BoolField f, BoolField res)**: Store the value of t (resp. f) in res where cond is true (resp. false).